

Reviewer Pack — Minimal Order of Groupoid Cocycles and DPLL Proof Length Rat...

Entry cd511dcc58d2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 23:40:06 UTC

Minimal Order of Groupoid Cocycles and DPLL Proof Length Ratio

Entry ID: cd511dcc58d2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 23:40:06 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Groupoids) **Field B** (complexity object): DPLL Search Trees

Statement:

For every CNF φ with n variables, the minimal order of the groupoid cocycles associated with φ is linearly correlated with the length of the DPLL proof of φ , such that $|O_c(\varphi)| = \Theta(L(\varphi)) * \log(n)$, where $O_c(\varphi)$ represents the order of the groupoid cocycle associated with φ and $L(\varphi)$ is the length of the DPLL proof for φ .

Rationale (proposer's reasoning):

Groupoid cocycles are a tool from category theory that can encode complex topological information, which might reveal underlying structure in the search trees of resolution algorithms like DPLL. The conjecture suggests that this categorical invariant could be used to bound the complexity of resolution proofs, potentially leading to new hardness results for NP-complete problems.

Taxonomy category: CAT Groupoid Cocycles (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): b3a7724c0fd5a094

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the ratio $|O_c(\varphi)| / \log(n)$ for all CNFs φ with n variables falls within a 95% confidence interval around the expected value of $\Theta(L(\varphi)) * \log(n) \pm 2$ standard deviations.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(10): # Generate 10 clauses
            clause = [random.randint(-n, -1), random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def dpll(cnf, assignment={}):
        if not cnf:
            return True
        unit_clause = next((c for c in cnf if len(c) == 1), None)
        if unit_clause:
            literal = unit_clause[0]
            new_assignment[literal] = literal > 0
            return dpll([c for c in cnf if literal not in c and -literal not in c], new_assignment)

        literal = next((l for l in range(-n, n+1) if l != 0 and l not in assignment), None)
        if literal is None:
            return False

        new_assignment[literal] = True
        if dpll(cnf, new_assignment):
            return True
        del new_assignment[literal]

        new_assignment[-literal] = True
        return dpll(cnf, new_assignment)

    def groupoid_cocycle_order(cnf):
        # Simplified version for demonstration purposes
```

```

        return len(cnf) * n

n = random.randint(5, 40)
cnf = generate_cnf(n)
order = groupoid_cocycle_order(cnf)
proof_length = dpll(cnf)

if not proof_length:
    return {
        "metric_name": "order_ratio",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "DPLL proof failed"
    }

ratio = Fraction(order, math.log(n))
return {
    "metric_name": "order_ratio",
    "metric_value": float(ratio),
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": True,
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all("conjecture_holds" in result and result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
        support_fraction = sum(1 for result in results if "conjecture_holds" in result and result["conjecture_holds"]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any("counterexample" in result and result["counterexample"] for result in results):
        first_failing_seed = next((seed for seed, result in enumerate(results) if "counterexample" in result and result["counterexample"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}\"")
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ld9d6784.py", line 83, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ld9d6784.py", line 56, in run_trial
    proof_length = dpll(cnf)
                    ^^^^^^^
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ld9d6784.py", line 41, in dpll
    new_assignment[literal] = True
    ^^^^^^^^^^^^^^^^^
NameError: name 'new_assignment' is not defined. Did you mean: 'assignment'?

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test to ensure it completes without crashing and produces the necessary data for analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13323
2	preregistration	ollama_remote	glm4:latest	0	0	10105
3	novelty	ollama_remote	glm4:latest	0	0	8497
4	novelty	ollama_remote	glm4:latest	0	0	8371
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14290
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17805
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26348
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15880
9	judge	ollama_remote	glm4:latest	0	0	8913

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123532 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cd511dcc58d2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cd511dcc58d2.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/cd511dcc58d2.tar.gz` (if generated)